

Qiskit vs PyTket IBM Torino QFT Roundtrip Benchmark Report

Benchmark suite v1.0

February 25, 2026

1 Experiment Specification (per `llm.json`)

1.1 Experiment identity

- **Experiment name:** `qiskit_vs_pytket_ibm_torino_qft_roundtrip`
- **Version:** 1.0
- **Backend:** `ibm_torino` (provider: `ibm_quantum`)
- **Execution mode:** real hardware only; sequential runs.

1.2 Circuit family (abstract definition)

Circuit is defined by the following build steps:

1. **X_ALL** — Apply X gate to every qubit.
 2. **QFT** — Apply QFT over all qubits (framework chooses decomposition).
 3. **IQFT** — Apply inverse QFT over all qubits.
 4. **MEASURE_ALL** — Measure all qubits into classical bits.
- **Expected ideal output bitstring:** `1...1` (all ones).
 - **Bitstring convention:** Framework-native then normalized to `msb_left`.

1.3 Frameworks and compilation

- **Qiskit:** optimization levels 0–3; routing: framework default for backend; transpiler optimization_level preset.
- **PyTket:** optimization levels 0–3; routing: framework default for backend; preset levels mapped to Qiskit 0–3.

1.4 Metrics (per `llm.json`)

1.4.1 Post-routing circuit metrics

- **Depth:** Circuit depth after mapping/routing to backend topology.
- **Two-qubit gate count:** Count of all 2Q gates after decomposition and routing. Gate set preference: `cx`, `ecr`, `cz`, `swap`, `rzz`, other 2Q if present; per-gate breakdown recorded.

1.4.2 Hardware result metrics

- **Population target state:** Probability mass of the expected all-ones bitstring after normalization.
- **Hellinger fidelity:** Fidelity between observed distribution p and ideal distribution q (delta at all-ones):

$$F_H = 1 - \frac{1}{\sqrt{2}} \sqrt{\sum_i (\sqrt{p_i} - \sqrt{q_i})^2}, \quad q_{\text{target}} = 1, \quad q_{\text{else}} = 0.$$

Counts are normalized over all observed bitstrings.

1.4.3 Runtime metrics

- **compile_time_seconds:** Mean wall-clock transpilation time (time.perf_counter).
- **execution_time_seconds:** Wall time from job submission to completion.
- **peak_tracemalloc_mib:** Peak tracemalloc memory during compilation (resident-set proxy).

1.5 Aggregation and primary statistic

Per llm.json: report **mean, std, median; primary stat for tables: mean.**

1.6 Reproducibility

- Fixed random seed: 12345 (circuit deterministic; seed affects toolchain internals).
- System info and config recorded in metadata.

2 Run configuration used

This report uses the quick configuration (subset of full llm.json grid):

- **Qubit counts:** 5
- **Optimizer level:** 0 (both frameworks)
- **Shots per run:** 512
- **Repetitions per configuration:** 1
- **Backend properties SHA256:** 598197e9424d1e8e66a78b3ea2b757c0764275fccf68d90b6428d2da96b3e7d (same for both frameworks; calibration consistency).

2.1 System and metadata

- **Platform:** Windows-11-10.0.26200-SP0
- **Python:** 3.12.10
- **Timestamp (UTC):** 2026-02-25T05:40:56
- **Framework versions:** Qiskit 2.3.0; PyTket 2.13.0

3 Per-run records (result schema)

Per `llm.json` `per_run_record_fields`: `timestamp_utc`, `framework`, `framework_version`, `backend_name`, `backend_properties_sha256`, `qubit_count`, `optimizer_level`, `shots`, `repeat_index`, `compile_time_seconds`, `peak_tracemalloc_mib`, `post_routing_depth`, `post_routing_2q_total`, `post_routing_2q_breakdown`, `execution_time_seconds`, `population_target_state`, `hellinger_fidelity`, `calibration_match_with_paired_framework`, `notes`.

Table 1: Per-run records (primary runtime and circuit metrics).

Framework	Backend	n_{qubits}	opt	Shots	Compile (s)	Exec (s)	Depth	2Q total
Qiskit	ibm_torino	5	0	512	1.971	7.062	352	106
PyTket	ibm_torino	5	0	512	0.426	6.186	250	78

Table 2: Per-run memory and hardware result metrics.

Framework	Peak tracemalloc (MiB)	Population target	Hellinger fid.	Calibration match
Qiskit	7.10	0.2285	0.2775	yes
PyTket	0.638	0.4707	0.4397	yes

3.1 Post-routing two-qubit gate breakdown

Per `llm.json` `two_qubit_gate_set_preference` and per-gate breakdown:

- **Qiskit** (opt=0): `cz` = 106 (total 106).
- **PyTket** (opt=0): `CZ` = 78 (total 78).

3.2 Calibration consistency

`backend_properties_sha256` is identical for both runs (598197e9...), so calibration match with paired framework is satisfied for this report.

Note: PyTket circuits were compiled with PyTket then converted to Qiskit and executed via the Qiskit Runtime Sampler (job mode) for compatibility with the IBM open plan; compilation metrics and 2Q breakdown are from PyTket, execution time is wall time of the Qiskit Sampler run.

4 Per-configuration aggregates (primary stat: mean)

Per `llm.json` `per_configuration_aggregate_fields`: `framework`, `backend_name`, `qubit_count`, `optimizer_level`, `shots`, `n_repeats`, `mean_depth`, `mean_2q_total`, `mean_compile_time_seconds`, `mean_execution_time_seconds`, `mean_peak_tracemalloc_mib`, `mean_population_target_state`, `mean_hellinger_fidelity`, `std_*`, `median_*`.

Table 3: Aggregates (primary statistic: mean).

Framework	n_{qubits}	opt	mean depth	mean 2Q	mean compile (s)	mean exec (s)	mean MiB	mean pop.	mean F_H
Qiskit	5	0	352	106	1.971	7.062	7.10	0.2285	0.2775
PyTket	5	0	250	78	0.426	6.186	0.638	0.4707	0.4397

With $n_{\text{repeats}} = 1$, `std` and `median` equal the single-run values; reported `std` = 0, `median` = same as `mean` for compile and execution time.

5 Figures

All plots use the primary statistic (mean) per llm.json aggregation. Data: $n_{\text{qubits}} = 5$, optimizer level 0, 512 shots, ibm_torino.

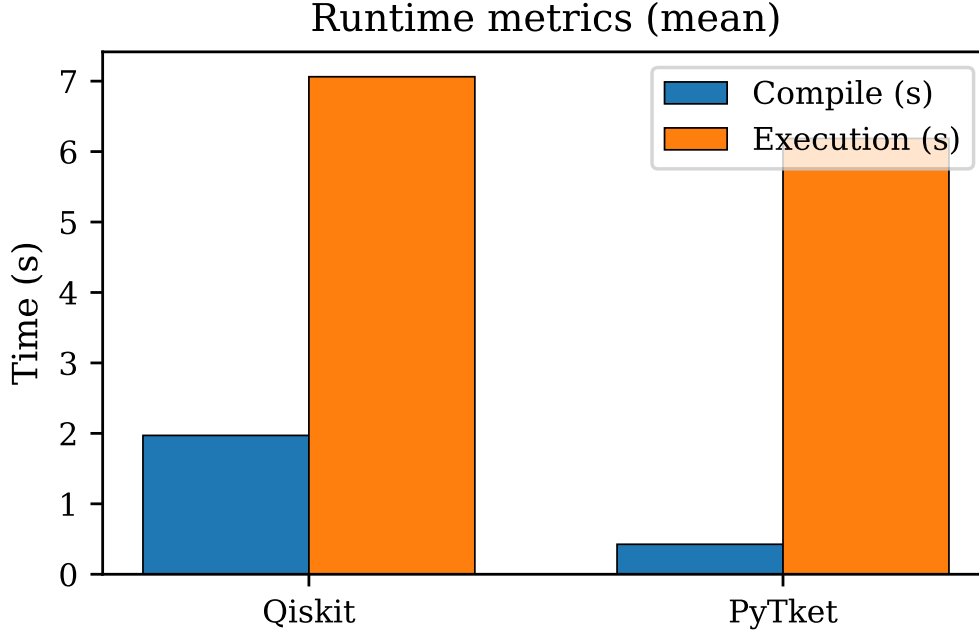


Figure 1: Runtime metrics: mean compile time (s) and mean execution time (s) by framework.

6 Normalized counts (target bitstring)

Expected ideal outcome: 11111 (msb_left, $n_{\text{qubits}} = 5$).

- **Qiskit:** $P(11111) = 0.2285$.
- **PyTket:** $P(11111) = 0.4707$.

Full normalized count distributions are stored in `raw_runs_quick.jsonl` and `benchmark_results_quick.js` (counts_normalized).

7 Summary

1. Experiment and metrics follow `llm.json`: circuit family (X_ALL, QFT, IQFT, MEASURE_ALL), backend `ibm_torino`, bitstring convention `msb_left`, expected output all-ones.
2. Post-routing circuit metrics: depth and two-qubit gate count (with breakdown) reported per run and as mean over repeats.
3. Hardware result metrics: population on target state and Hellinger fidelity reported; formula and normalization as in `llm.json`.
4. Runtime metrics: compile time (wall-clock), execution time (submission to completion), peak tracemalloc (MiB) reported; aggregation uses mean as primary stat.

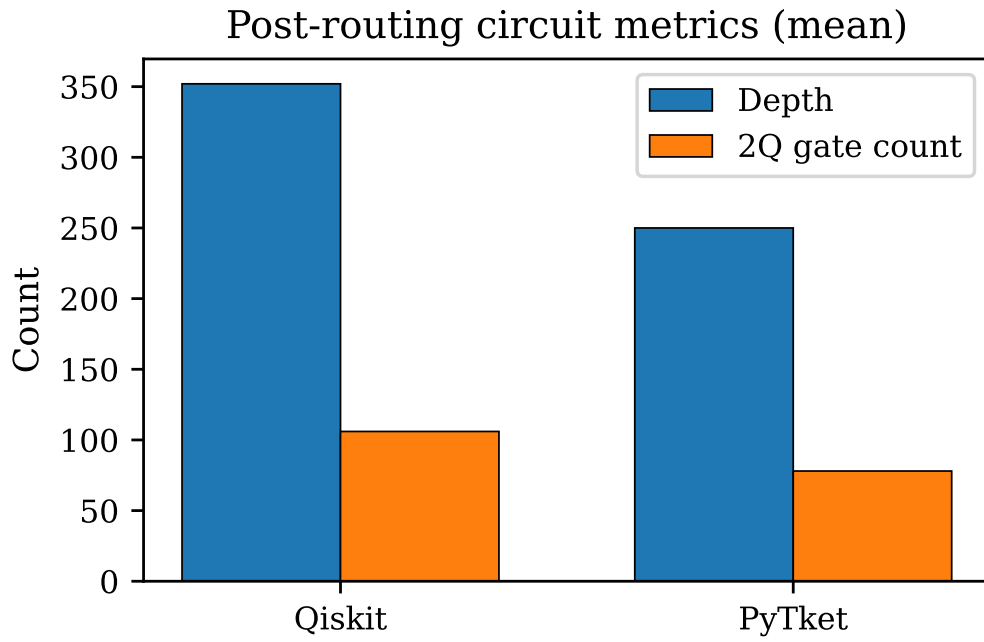


Figure 2: Post-routing circuit metrics: mean depth and mean two-qubit gate count by framework.

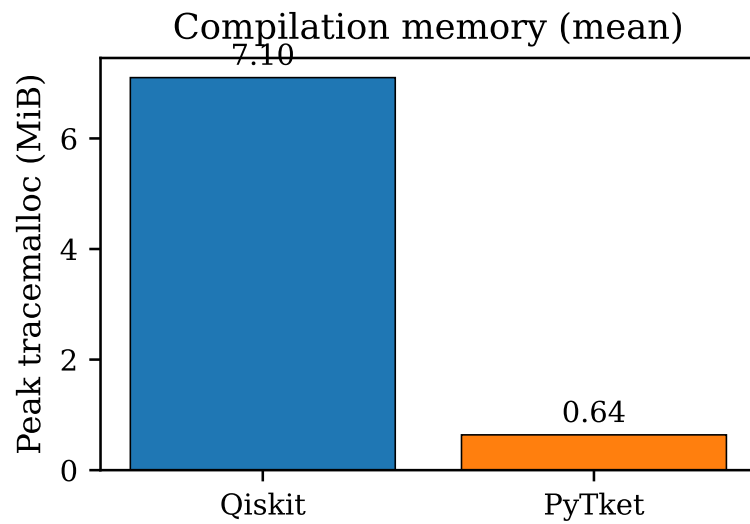


Figure 3: Compilation memory: mean peak tracemalloc (MiB) by framework.

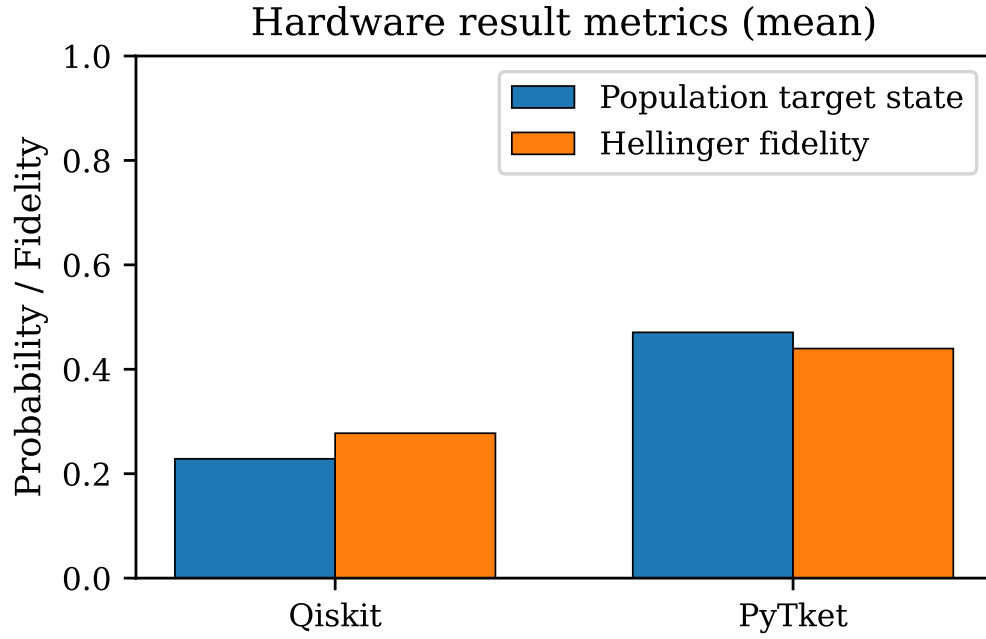


Figure 4: Hardware result metrics: mean population on target state (all-ones) and mean Hellinger fidelity by framework.

5. Calibration consistency: same `backend_properties_sha256` for both frameworks; `calibration_match_with` = true.
6. Reproducibility: fixed seed 12345; system info and config recorded in metadata.